

13.3 Il guscio convesso della PLI

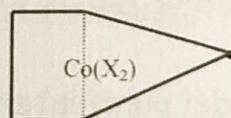
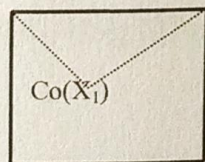
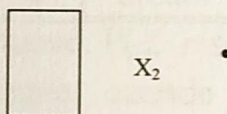
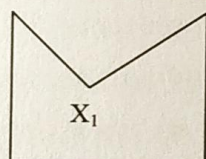
Risulta abbastanza ovvio verificare come la PLI, a differenza della PL non rientri nella classe dei problemi di Programmazione Convessa. Infatti la regione ammissibile della PLI è data dal politopo (o poliedro) definito da un insieme di vincoli lineari e dalle restrizioni di positività e di interezza sulle variabili. Quindi è un reticolo di punti distanti tra loro che certamente non costituisce un set convesso !

Viceversa la regione ammissibile del rilassamento lineare della PLI definita da vincoli lineari e dalle restrizioni di positività sulle variabili è un set convesso.

Diamo ora la seguente definizione :

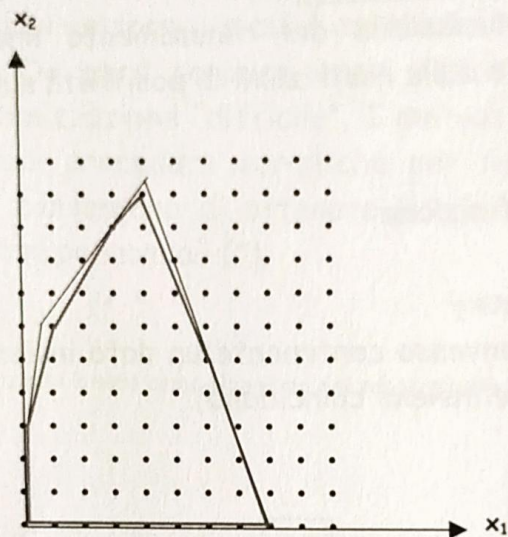
Convex hull (guscio convesso)

È il più piccolo insieme convesso contenente un dato insieme (se l'insieme contenuto è convesso i due insiemi coincidono).



Esempio di set concavi : X_1 e X_2 e loro gusci convessi $co(X_1)$ e $co(X_2)$

Consideriamo nuovamente l'esempio visto :



Possiamo considerare 3 regioni :

- La regione ammissibile del rilassamento lineare (il politopo esterno),
- La regione ammissibile del PLI (i punti interi contenuti nel politopo esterno)
- Il *guscio convesso* dei punti interi ammissibili.

E' chiaro, per quanto detto in precedenza, che operando con il simplesso sul politopo esterno si ottiene un'approssimazione più o meno buona dell'ottimo del PLI (lower bound).

Viceversa se, con la stessa procedura, si opera sul politopo interno si ottiene sempre l'ottimo del PLI originale. Infatti il guscio convesso di tutti i punti interi ammissibili è un politopo (o poliedro) convesso che ha

per vertici solo punti interi ammissibili e l'ottimo si trova sempre in almeno una soluzione ammissibile di base !

Purtroppo, il passaggio dal Politopo esterno a quello interno non è affatto banale !

In generale nella soluzione di un PLI ha grande importanza la determinazione di upper e lower bound il più possibile vicini all'ottimo (bound efficienti) che possono essere ottenuti con adeguati rilassamenti del problema originario.

Le categorie principali di rilassamenti utilizzate sono le seguenti :

a) Rilassamento lineare

Si ottiene eliminando i vincoli di interezza sulle variabili decisionali e risolvendo con il simplesso (o con altro metodo) il PL ottenuto.

b) Rilassamento per eliminazione

Si ottiene eliminando da $Ax \geq b$ alcuni vincoli denominati " side constraints " (*) e ottenendo un nuovo PLI risolubile con un algoritmo polinomiale efficiente (ad esempio quando il PLI è del tipo assegnamento, shortest path , spanning tree etc..)

c) Rilassamento lagrangiano

Come il precedente, in cui però i vincoli rimossi vengono poi penalizzati lagrangianamente introducendoli nella funzione obiettivo.

(*) come è stato già accennato si definiscono "side constraints" quei vincoli che rendono "intrattabile" un problema che, altrimenti, sarebbe "matematicamente trattabile" (cioè risolubile con algoritmi polinomiali di bassa potenza).

d) Introducendo ulteriori vincoli che siano ridondanti per il politopo che definisce la regione ammissibile del PLI. Tali disuguaglianze sono anche dette "**tagli**", perché esse tagliano via parte del Politopo, lasciando però tutti i punti interi ammissibili contenuti in esso, consentendo così di arrivare a bounds più "stretti" (cioè più vicini all'ottimo).

Un esempio banale è dato in un problema del tipo :

$$\begin{aligned} \text{Min } & cx \\ & Ax \geq b \\ & x \in \{ 0, 1 \} \end{aligned}$$

Eliminando i vincoli di binarietà, ma aggiungendo i vincoli (**tagli di Dantzig**):

$$0 \leq x \leq 1$$

ridondanti nella formulazione originaria si ottiene un bound migliore rispetto a quello che si otterrebbe non introducendo tale vincolo (In tal caso alcune variabili potrebbero anche assumere valori maggiori di 1 !)

In generale, tale problema viene risolto con il simplesso e, rispetto al caso di semplice rimozione dei vincoli di interezza, si ottiene un lower bound maggiore (o un upper bound minore).

e) Rilassamento surrogato

Combinando linearmente un insieme di vincoli ("surrogando") in un unico vincolo lineare. Quindi, in linea di principio, per un qualsiasi PLI un lower bound viene ottenuto risolvendo un altro PLI denominato **Knapsack** che si ottiene combinando linearmente tutti i vincoli del PLI originale. Il knapsack, come si vedrà in seguito è un PLI costituito da un solo vincolo.

- Un upper bound in un problema di minimizzazione (oppure un lower bound in un problema di massimizzazione) viene ottenuto considerando il costo di una qualsiasi soluzione ammissibile x (ad esempio ottenuta mediante un algoritmo euristico)

13.4 Problemi "facili" di PLI

Esiste una categoria di PLI per cui il Rilassamento lineare non solo dà una stima sull'ottimo, ma dà l'ottimo effettivo e costituisce, comunque, un approccio alla soluzione di PLI più complicati.

Definizioni e principali risultati

Definizione 1

Una matrice quadrata, ad elementi interi, B è detta unimodulare (UM) se $\det B = \pm 1$

Definizione 2

Una matrice A di m righe ed n colonne è totalmente unimodulare (TUM) se ogni sua sottomatrice quadrata non singolare è UM.

E' possibile, con procedimento analogo a quello seguito per la matrice dei trasporti, provare il seguente risultato :

Teorema : Se A è TUM allora tutti i vertici del politopo X hanno coordinate intere ove X sia così definito :

$$X = \{ x : Ax = b, x \geq 0, b \text{ intero} \}$$

Si noti che si tratta di una condizione sufficiente ma non necessaria.

Teorema:

Una matrice intera A con elementi: $0, \pm 1$ è TUM se ogni colonna ha non più di due elementi non nulli e l'insieme I delle sue righe può essere partizionato in due insiemi I_1 e I_2 disgiunti, per cui $I_1 \cup I_2 = I$ e tali che:

- 1) Se una colonna ha 2 elementi dello stesso segno le righe relative sono in insiemi differenti (I_1 e I_2).
- 2) Se una colonna ha due elementi di segno diverso le righe relative sono nello stesso insieme (cioè $I_1 \equiv I$ e $I_2 = \emptyset$).

Esempio 1:

$$\text{Min } c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 + c_5x_5 + c_6x_6$$

$$x_1 + \quad \quad \quad x_4 \quad \quad \quad = b_1$$

$$\quad x_2 + \quad \quad \quad x_5 \quad \quad \quad = b_2$$

$$\quad \quad x_3 + \quad \quad \quad x_6 = b_3$$

$$x_1 + \quad \quad \quad x_4 \quad \quad \quad = b_4$$

$$\quad x_2 + \quad \quad \quad x_5 \quad \quad \quad = b_5$$

$$\quad \quad x_3 + \quad \quad \quad x_6 = b_6$$

$$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0$$

Esempio 2:

$$\text{Min } c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 + c_5x_5 + c_6x_6$$

$$x_1 + \quad \quad \quad = b_1$$

$$-x_1 \quad x_2 + \quad \quad \quad = b_2$$

$$\quad -x_2 \quad x_3 \quad \quad \quad = b_3$$

$$\quad \quad -x_3 \quad +x_4 \quad \quad \quad = b_4$$

$$\quad \quad \quad -x_4 \quad +x_5 \quad \quad \quad = b_5$$

$$\quad \quad \quad \quad -x_5 \quad +x_6 \quad \quad \quad = b_6$$

$$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0$$

13.5 Soluzione generale della PLI

Esclusa la classe di PLI con matrici di vincoli TUM, il problema della PLI è un problema aperto, nel senso che non è stato definito un algoritmo generale di soluzione (come, ad esempio, il simplesso per la PL) valido, almeno in linea teorica, per tutte le istanze.

L'unico modo efficace di risolvere PLI di grandi dimensioni è quello di far ricorso a metodi euristici.

La definizione di procedure euristiche può essere basata su criteri empirici, ispirati dal particolare problema in studio ("rules of thumb"), ma anche influenzata dalla struttura (simmetria, tipo di vincoli etc..) della sua formulazione matematica.

Per problemi di dimensioni contenute si possono definire delle strategie di soluzione esatte.

- Metodi enumerativi

A differenza di un PL, un PLI ha un numero finito di soluzioni ammissibili (ad esempio, se il max. valore che una variabile può assumere è N , un upper bound sul numero di punti ammissibili è $(N + 1)^n$).

Ovviamente l'enumerazione esplicita di tutte le soluzioni ammissibili è proibitiva!

E' possibile, però, definire delle strategie di soluzione che si basino su un'enumerazione implicita piuttosto che esplicita, per cui, cioè, sia sufficiente enumerare solo un campione, in base a qualche determinato criterio, di tutte le soluzioni ammissibili.

Tra queste tecniche particolare rilevanza hanno gli algoritmi branch and bound e le procedure basate sulla programmazione dinamica.

- Metodi poliedrali

Tali metodi tendono ad individuare od approssimare la *convex hull* del PLI considerato ed operare poi con algoritmi di soluzione della PL.

La prima classe di metodi definita è stata quella dei piani di taglio (*cutting planes*).

Una categoria di algoritmi che negli ultimi anni ha permesso di ottenere soluzioni soddisfacenti (solo però per alcune classi specifiche di PLI) è quella denominata :

- Branch and bound and cutting planes o più brevemente

Branch and cut

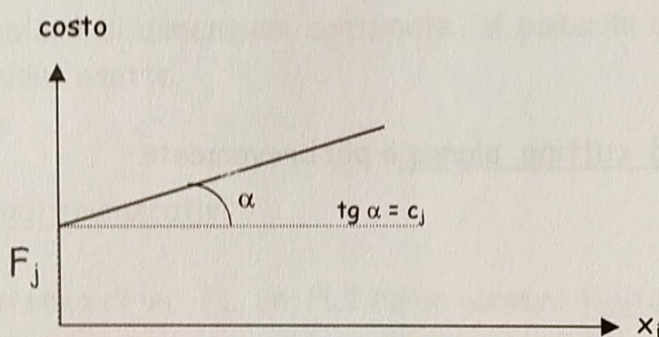
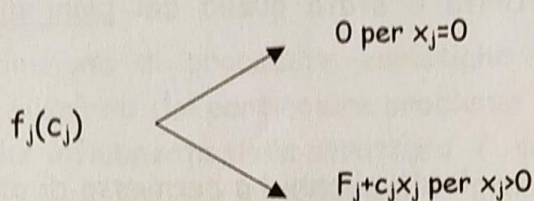
Si tratta sempre di metodi enumerativi, in cui, però, nel politopo definito dal rilassamento lineare del problema vengono aggiunti tagli che lo restringono in modo efficace (facce o faccette del politopo o tagli che le approssimino). La soluzione ottima del politopo così ristretto viene sfruttata come bound in una strategia generale branch and bound.

13.6 Formulazioni matematiche di alcuni problemi classici della PLI

1. Problema relativo a costi di avviamento

Non sempre un costo lineare cx costituisce un'accettabile approssimazione di un costo reale.

Spesso il costo in un problema di produzione può essere così definito :



Ove F_j è il costo di avviamento della produzione.

Il costo è quindi una funzione con un gradino e quindi non è lineare !

La funzione obiettivo può, però, essere linearizzata, introducendo un'ulteriore variabile decisionale y_j così definita:

$y_j = 0$, se $x_j = 0$, se cioè si decide di non produrre il bene j

$y_j = 1$, se $x_j > 0$, se cioè si decide di produrre il bene j

Occorre però introdurre dei vincoli che mettano in relazione le variabili decisionali y_j con le x_j , infatti vi è un legame tra le due variabili per cui :

$$y_j = 0 \rightarrow x_j = 0$$

Ciò può essere modellato introducendo i seguenti vincoli :

$$x_j \leq My_j$$

Si assume poi $M \rightarrow \infty$ in modo che se $y_j = 1$, i vincoli $x_j \leq M$ divengono ridondanti.

Con questi accorgimenti possiamo introdurre una funzione obiettivo lineare in cui sono tenuti in conto anche i costi di avviamento :

$$z = \sum_{j=1}^n c_j x_j + \sum_{j=1}^n F_j y_j$$

Considerando gli eventuali vincoli sulle risorse $Ax = b$, possiamo definire il problema come un problema di programmazione mista in cui l'unica non linearità è associata all'interezza delle y_j :

Min z

$$z = \sum_{j=1}^n c_j x_j + \sum_{j=1}^n F_j y_j$$

$$Ax = b$$

$$x_j \leq My_j, \quad j = 1, 2, \dots, n$$

$$x_j \geq 0, y_j \in \{0, 1\}, \quad j = 1, 2, \dots, n$$

2. Problema dei lotti di produzione

Il problema è quello di decidere la produzione ottimale di n lotti con il limite che se x_j rappresenta la quantità incognita del prodotto j la sua produzione può essere avviata solo se supera un certo lotto minimo L_j .

Quindi se $x_j < L_j \rightarrow x_j = 0$
 se $x_j \geq L_j \rightarrow x_j \geq L_j \geq 0$

Si tratta quindi di un vincolo dicotomico, chiaramente non lineare :

or $x_j = 0$ or $x_j \geq L_j$

Per evitare tale non linearità anche in questo caso ad ogni variabile x_j viene associata una variabile binaria y_j nel modo seguente :

$y_j = 1$ se il bene j è prodotto
 $y_j = 0$ se il bene j non è prodotto

Per mettere in relazione tali variabili introduciamo i seguenti vincoli addizionali :

- $x_j \leq M y_j$ tali vincoli impediscono che, nel caso si verifichi $y_j = 0$, risulti $x_j > 0$, inoltre se $y_j = 1$, poiché $M \rightarrow \infty$, i vincoli divengono ridondanti,
- $x_j \geq L_j y_j$ tali vincoli, per ogni $y_j = 1$, assicurano che x_j sia almeno uguale ad L_j , se invece risulta $y_j = 0$ tali vincoli diventano ridondanti

In tal modo si riesce a formulare il problema dei lotti di produzione come un problema di programmazione mista :

Min z

$$z = \sum_{j=1}^n c_j x_j$$

$$Ax = b$$

$$x_j \leq M y_j ,$$

$$x_j \geq L_j y_j ,$$

$$x_j \geq 0, y_j \in \{0, 1\}$$

$$j = 1, 2, \dots, n$$

$$j = 1, 2, \dots, n$$

$$j = 1, 2, \dots, n$$

3. Il problema dello zaino (Knapsack)

"Si deve riempire uno zaino di volume V e sono disponibili n oggetti. Ogni oggetto j ($j=1,2,\dots,n$) ha un certo volume v_j ed un valore associato c_j . Ogni oggetto è disponibile in più unità.

Gli oggetti vanno scelti in modo da non superare il volume V dello zaino ed in modo che il valore complessivo degli oggetti introdotti nello zaino sia il massimo possibile."

Si definisce una variabile decisionale binaria :

$x_j \rightarrow$ numero di unità dell'oggetto j nello zaino

Il knapsack può quindi essere formulato come PLI nel modo seguente :

$$\text{Max } \sum_{j=1}^n c_j x_j$$

$$\sum_{j=1}^n v_j x_j \leq V$$

$$x_j \geq 0 \text{ e intero}$$

Questo problema ha una variante nota come "Knapsack binario" in cui si considera che ogni oggetto sia disponibile in un'unica unità. La formulazione è la stessa, cambia invece la variabile decisionale che diviene

$x_j \rightarrow 1$ se l'oggetto j è in soluzione, $x_j \rightarrow 0$ se l'oggetto j non è in soluzione

Il Knapsack è in generale un problema interessante in quanto pur essendo difficile, ha un grado di difficoltà inferiore agli altri problemi di PLI. Infatti pur non essendo risolubile con algoritmi polinomiali è risolubile con algoritmi pseudopolinomiali. Tali algoritmi hanno una complessità computazionale che cresce esponenzialmente con le dimensioni delle istanze ma che, per certi valori dei parametri che le caratterizzano, risulta praticamente polinomiale.

Per il Knapsack esiste una classica procedura euristica di soluzione che possiamo riassumere nel modo seguente :

1. Disporre gli oggetti in una lista secondo l'ordine non crescente del rapporto $\{c_j/v_j\}$
2. Introdurre gli oggetti, uno alla volta, nello zaino secondo tale ordine. Se non è possibile inserire un oggetto perché viene violato il vincolo si passa all'oggetto successivo secondo l'ordine stabilito sino a che la lista viene svuotata.

E' facile vedere con un esempio banale come tale procedura possa non condurre all'ottimo

Esempio

$$\text{Max } 60x_1 + 40x_2$$

$$6x_1 + 5x_2 \leq 10$$

x_1, x_2 interi positivi

procedura euristica :

1. $c_1/v_1 = 10$; $c_2/v_2 = 8$
2. pongo $x_1 = 1$, resta il volume da riempire $V=10-6=4$. E' evidente che non si possono aggiungere né altre unità dell'oggetto 1 né unità dell'oggetto 2 perché il vincolo verrebbe violato.

La soluzione data dalla procedura euristica è quindi $x=(1,0)$, $z=60$

La soluzione ottima è però differente :

$$x^*=(0,2) \text{ con } z^*=80$$

3. Set covering e Set partitioning

Si tratta di formulazioni matematiche ricorrenti nella PLI con cui possono essere modellati molti problemi.

Consideriamo ad esempio il seguente problema di localizzazione:

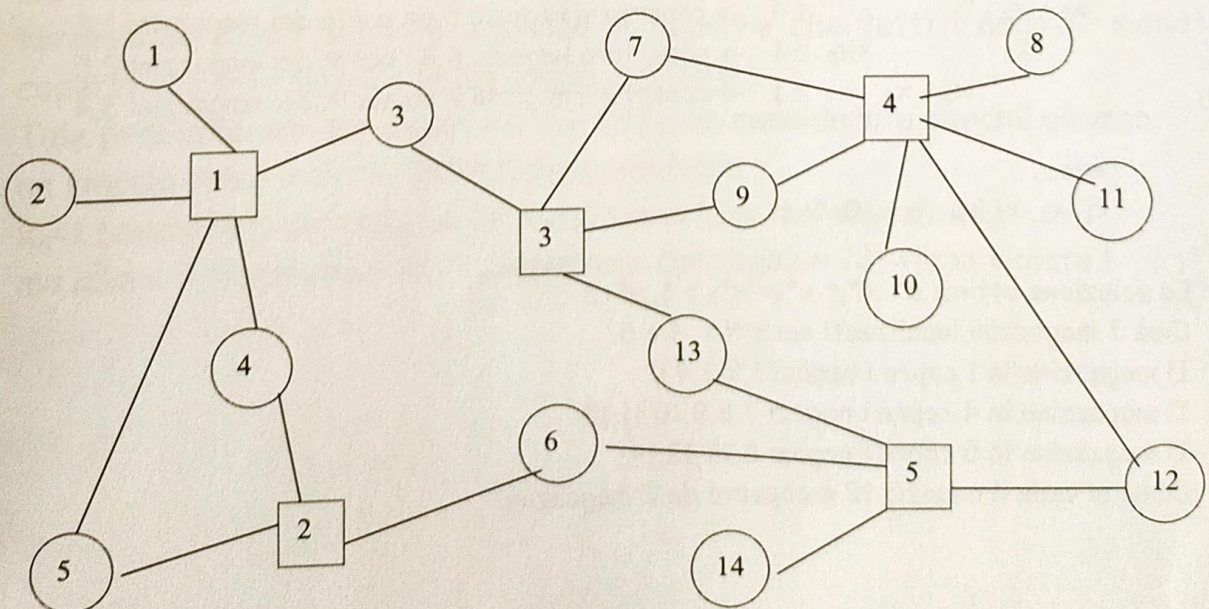
"Un'azienda dispone di m punti vendita che possono essere riforniti giornalmente da magazzini secondo criteri dettati dalla distanza dai magazzini e dal tipo di merci richieste dai punti vendita. I magazzini possono essere localizzati in $n < m$ possibili siti. Si vuole definire quella localizzazione dei magazzini che coprendo tutti i negozi ne minimizza il numero"

In base al tipo di merci fornite ed alle distanze magazzini-punti di vendita è noto, per ciascun punto vendita, da quale magazzino potenziale esso possa essere rifornito. Tali dati possono essere forniti mediante una matrice di copertura ("covering") $A = \{a_{ij}\}$ tale che ogni suo generico elemento a_{ij} sia binario e definito nel modo seguente :

$a_{ij} \rightarrow 1$ se il punto di vendita i può essere coperto da un magazzino situato nella localizzazione j

$a_{ij} \rightarrow 0$ in caso contrario.

Nella figura seguente viene rappresentato un esempio di possibili coperture di 14 punti vendita da 5 possibili localizzazioni potenziali :



Una tale situazione viene rappresentata dalla matrice A in cui vengono attribuiti i valori opportuni agli elementi a_{ij} .
 Ad esempio, nell'istanza in figura, per il magazzino 1 si avrà :
 $a_{11}=a_{21}=a_{31}=a_{41}=a_{51}=1$, mentre $a_{61}=a_{71}=...=a_{14,1}=0$
 etc..

La formulazione di un tale problema di copertura ottima sarà il seguente :

$$\text{Min} \sum_{j=1}^5 x_j$$

$$\sum_{j=1}^5 a_{ij} x_j \geq 1, \quad i = 1, 2, \dots, 14$$

$$x_j \in \{0, 1\}$$

In forma esplicita e per la particolare istanza considerata:

$$\text{Min } x_1 + x_2 + x_3 + x_4 + x_5$$

$$x_1 \geq 1 \rightarrow \text{copertura negozio 1 da parte del magazzino 1}$$

$$x_1 \geq 1 \rightarrow \text{copertura negozio 2 da parte del magazzino 1}$$

$$x_1 + x_3 \geq 1 \rightarrow \text{copertura negozio 3 da parte dei magazzini 1,3}$$

$$x_1 + x_2 \geq 1 \rightarrow \text{copertura negozio 4 da parte dei magazzini 1,2}$$

$$x_1 + x_2 \geq 1 \rightarrow \text{copertura negozio 5 da parte dei magazzini 1,2}$$

$$x_2 + x_5 \geq 1 \rightarrow \text{copertura negozio 6 da parte dei magazzini 2,5}$$

$$x_3 + x_4 \geq 1 \rightarrow \text{copertura negozio 7 da parte dei magazzini 3,4}$$

etc....

$$x_1, x_2, x_3, x_4, x_5 \in \{0, 1\}$$

La soluzione ottima è : $x_1^* = x_4^* = x_5^* = 1$, $z^* = 3$.

Cioè 3 magazzini localizzati nei siti 1, 4 e 5.

Il magazzino in 1 copre i negozi 1,2,3,4,5

Il magazzino in 4 copre i negozi 7,8,9,10,11,12

Il magazzino in 5 copre i negozi 6,12,13,14.

Come si vede il negozio 12 è coperto da 2 magazzini.

In generale se si vuole minimizzare il costo complessivo associato alla localizzazione dei magazzini, la formulazione sarà quella classica di tutti i problemi di set covering :

$$\text{Min} \sum_{j=1}^n c_j x_j$$

$$\sum_{j=1}^n a_{ij} x_j \geq 1, \quad i = 1, 2, \dots, n$$

$$x_j \in \{0, 1\}, \quad j = 1, 2, \dots, n$$

Esiste nella PLI una classe di problemi che hanno una formulazione simile, ad eccezione dei vincoli, che sono, in quest'ultimo caso, di stretta uguaglianza :

$$\text{Min} \sum_{j=1}^n c_j x_j$$

$$\sum_{j=1}^n a_{ij} x_j = 1, \quad i = 1, 2, \dots, n$$

$$x_j \in \{0, 1\}, \quad j = 1, 2, \dots, n$$

Tale formulazione è nota come Set Partitioning.

Ad esempio, se nel caso precedente fosse stata imposta tale formulazione, ciò sarebbe equivalso a imporre che tutti i negozi siano coperti, ma ogni negozio da un solo magazzino !.

Tale problema non ha soluzione per l'istanza considerata perché almeno un vincolo viene violato. Infatti deve risultare :

$x_4 = 1$ (copertura del negozio 8), $x_5 = 1$ (copertura del negozio 14)

ma allora il vincolo $x_4 + x_5 = 1$ (copertura del negozio 12) viene violato !